Design, Creation and Analysis of the W01-10-2 Group Warman Device

Theo Devos-Vernes (c3479858), MECH2110

Table of Contents

Table of Figures

Introduction

The Warman Design and Build Competition is a multi-university challenge, designed to teach groups of six students how to effectively design and build systems, evaluate prototypes, project manage and troubleshoot by having them build semi-autonomous devices to complete a given task. Each year group receives a different challenge, with this year's challenge (see Fig. 1 for an overview of the course) featuring a walled rock collection zone, a large ramp leading to the middle platform of a multi-level structure, and a hopper placed on the upper floor of the same structure. Depositing the rocks in the hopper would score maximum points per rock, while placing them on each progressively lower floor would yield fewer points, with the minimum points per rock (without dropping them) achieved by depositing them on the ground floor of the structure. Points would also be awarded for the device disturbing and collecting each rock, finishing in the correct area of the course, and the time taken to complete the course. The device was also required to meet a weight requirement of under 6kg and a volume requirement of at or under 400mm x 400mm x 400mm.

Our team decided on a strategy to aim for depositing all rocks on the top platform, with three major mechanisms: one for rock collection, one for transport, and one for rock deposit. This scope gave us the option of swapping out any mechanism if we wanted to aim for greater or fewer points. Our transport mechanism needed to be able to climb the curb made by the ramp's placement, ascend the gradient of the ramp, and have the capacity to rotate around its centre of mass. The mechanism for rock collection needed to be able to extend outwards from the device, in order to maintain the starting volume stated in the rules. The mechanism had to be able to extend to 360mm or greater to be able to collect all rocks, regardless of position within the collection zone. The rock deposit mechanism required the capabilities to extend upwards and slightly out of the device to be able to reach the extra height above the ramp to the upper platform, and deposit the rocks, while interfacing with the rock collection mechanism to ease the transfer of rocks from one mechanism to the other (i.e. a funnel system). We decided against aiming for the hopper as the mechanisms required to deposit in an area as small and positionally high seemed too complex given the time constraints of this project.

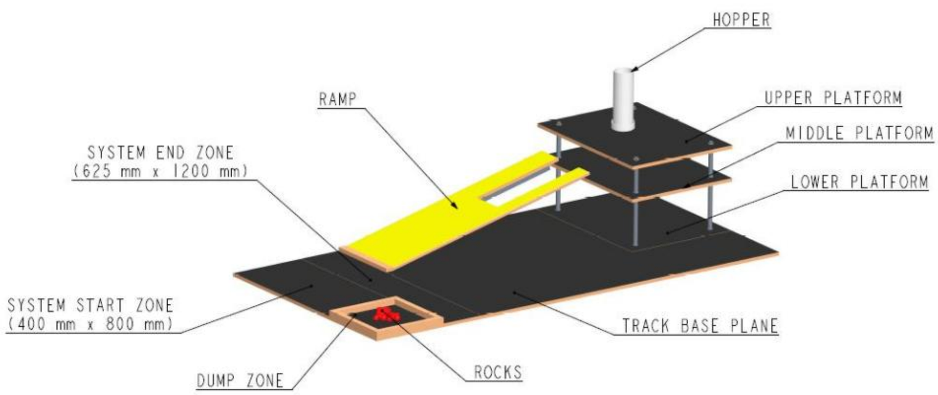


Figure 1. Schematic view of the 2026 Competition Track showing the track base plane, ramp, platforms, deposit zone, hopper, and end zone. Schematic obtained from [1]

Design Process

Initial Concept

The initial concept for the device was intentionally based on practical intuition rather than theoretical research. This would allow us to base our design on our own personal strengths, rather than constraining ourselves to within a purely theoretically framework. A preliminary concept for each of the major mechanisms will be detailed as follows:

For the transportation mechanism, given the scope and constraints listed in the introduction, we had two prevailing ideas: one continuous track on each side of the device (such as those found on all-terrain construction and military vehicles) or four omnidirectional wheels (popular in small-scale hobby devices), one placed on each side of the vehicle to allow control in all directions. Continuous tracks can negotiate rough terrain and have the most surface area (and therefor, grip) of the two options. Tracks are, however, expensive to purchase; or very time consuming to design and manufacture individual parts. Omnidirectional wheels excel in being easy to control, since each wheel can be controlled individually. However, available ones on the market within our price range lack enough grip to reliably make it up the ramp without modification. The final consensus was to use a set of continuous tracks, with two driven wheels and two idle wheels per side (see Fig. 2). The exact manufacturing method was left to the discretion of the group member tasked with creating it, detailed in the Manufacturing section.

The rock collection mechanism caused considerable debated amongst the group, because it would need to be custom designed. In the end, we decided that the mechanism would feature an excavator-inspired "boom and stick" system (see Fig. 3) on a rotating base, with servo motors replacing the hydraulic components of the system. This would allow the mechanism to "fold" in on itself to meet the starting volume restriction and to stretch outwards by the scoped 360mm simultaneously. To the end of the stick could be fitted any attachment we required, which would initially be a mud-bucket widened to width of the collection zone (pictured in Fig. 1) minus a margin of error we had yet to determine. This was designed using a CAD program (CREO Parametric 12) and 3D printed out of a material to be chosen based on testing (See Appendix A for a full gallery of CAD models for this mechanism).

The rock deposit mechanism was the most difficult to conceptualise, since a small mechanism that could complete the prior-scoped task would not be easy to design. A conveyor system that could pivot upwards was the best idea we could articulate. This had the capacity to cause some issues however, such as how to keep the rocks from rolling in place on the conveyor. These issues would need to be addressed by the group member assigned with creating the mechanism.

The initial plan to control this device would be to use any component-specific drivers or controllers (i.e. L298N Motor Driver) connected to one of two UNO R3 microcontrollers, which would then be connected to each other to work as one board with enough pins to satisfy our component choice. We intended every component bought to be running on either 5V or 12V power and planned to use two 2S lithium batteries (common in RC vehicles) running in parallel to output 14.8V and 30A of current. This would need to be split and stepped down to the required 5V and 12V, respectively.

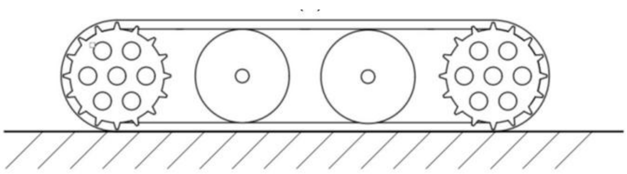


Figure 2. A simplified continuous track diagram with two drive wheels and two idle wheels. Diagram obtained from [2]

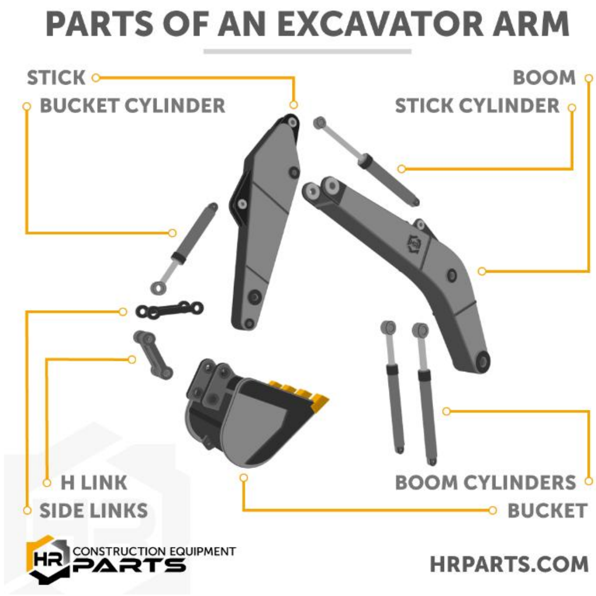


Figure 3. Diagram of the various parts of an excavator arm. Diagram obtained from [3]

Components and Hardware

Chassis:

- Plywood (6mm): Plywood was chosen as it is lightweight and easy to make necessary modifications to with common hand tools. Plywood is also electrically insulative, which avoids the need for extra measures to be taken against shorted circuits.

- Various Miscellaneous Bolts: These were chosen out of necessity, since the expense would rapidly climb if we were to buy every bolt to spec. Thus, we used a variety of found hardware to mount all our components. While not as aesthetically pleasing, it was – importantly – fully functional.

Excavator (Boom, Stick and Bucket):

- PLA Filament (3D Printed): PLA was chosen as the material as it is fast to print, easy to work with and is cheap. 3D printing appeared the best manufacturing method for the bulk of the mechanism as it allows rapid prototyping and high levels of dimensional accuracy.

- 6.4mm Aluminium Rod: This was chosen to function as a bearing surface for the joints of the Excavator mechanism, chosen for aluminium's softness, allowing for easy manufacturing of each bearing rod.
- DSS-P05 Servo: This servo was chosen for its convenient size (detailed in Final Solution), high lifting force of ~4kg on 5V power. Three would be needed in total.
- 24BYJ-48 Stepper Motor: This motor was chosen because we believed (incorrectly) that it had high torque and would thus be capable of rotating the entire Excavator assembly.

Continuous Tracks:

- Bicycle Chains: These were chosen for their strength and modifiability, with our group splitting the chains and reconnecting them at the required length.
- PVC Pipe (~15mm): This was to function as a friction surface for the tracks, chosen for its ability to be mounted to the bike chains in a way that provided enough grip to navigate the course.
- Mecanum Wheel with Motor: These were chosen as our drive wheels for their high torque, included gearbox, and included mounting system that was easily adapted to our needs. These wheels operate 5-10V; we intentionally opted to trade off risk to their lifespan for increased performance for this component and run them on 12V (and later, 14.8V). We would need four of these components in total.
- Miscellaneous Drive Belt Idlers and Wooden Dowel: These were mostly chosen for their availability to us, as one group member possessed several spare ones. Being a less critical part, the choice of material in this instance was considered less important. Drive Belt Idlers would be mounted to an appropriately sized wooden dowel to function as an axle.

Control System:

- UNO R3 Microcontroller: Chosen for the group's existing knowledge of the platform and compatibility with many components as they are extremely popular in hobby applications.

- 2S Li-Po Battery: Chosen for its popularity in hobby electronics, which made resources and support easy to find. The ability of this battery to provide 30A of current was also a major positive factor. Two of these batteries would be run in parallel to increase voltage output to 14.8V.
- Multi-Output Buck Converter: This would be necessary to convert the 14.8V output of the batteries to a 5V and 12V rail to power all other electrics.
- 10A Fast-Blow Fuse: Chosen as the calculation of currents for all other components made a 10A fuse the safe choice for our system while staying below the 30A output of our batteries. This meant the fuse would still blow if a short occurred.
- L298N Motor Driver: These are necessary to power all our motors and had the benefit of offering a 5V output that we could use if necessary. These drivers could also power and control the stepper motor required for the excavator system.

The rock deposit mechanism was intentionally excluded from the above list because it did not end up on our device. Reasons for this are detailed in 'Final Solution.'

Manufacturing

Most of our device was constructed using "DIY" methods, often incorporating common hand tools and found objects – an example of which being the car idlers we used. This was partially our choice, and partially a misinterpretation of the rules, specifically one about not using pre-purchased modules. Our literal interpretation of this aspect of the rules led to us making nearly everything, barring circuit boards, by hand. We split manufacturing up into segments, with each group member tasked with making one section of the whole device, the sections being: (1) chassis, (2) tracks, (3) excavator mechanism, (4) deposit mechanism, (5) electronics and coding. Our group had only five members which justifies the construction being split into five sections.

The chassis was made almost entirely out of plywood, which was cut down to a 360mm*300mm rectangle and a 360mm*150mm rectangle by a table saw. This would make up the two floors of which our device was comprised, with the upper floor being the smaller of the two. These would be spaced out by two offcuts of timber that were then cut to 100mm each. This was mounted using miscellaneous bolts that would go down through the entire ply-timber-ply beam and secured on the bottom with a nut.

The track itself was manufactured out of resized bicycle chains, a chain breaker was used for the resizing, with the final length being 800mm. The reason for this length is the size constrains of the entire device being 400mm. Since the track must travel twice that length plus the height of the track, 800mm was a logical length that allowed good margins for error. We used four of these chains, two per side to create the tracks. Small radius PVC pipe was then cut into 80mm lengths and then cut in half lengthways to make semi-circle treads. These were mounted concave-out onto two sets of chains using small bolts (see Fig. 4). Mecanum wheels were then inserted between the two chains (horizontally and vertically) so that it made adequate contact with the convex side of the PVC pipe so as to drive the tracks. The motor and gearbox assembly was then mounted to the underside of the lower floor of the chassis to make a nearly complete track assembly. Idler wheels were then mounted to a wooden dowel, suspended by U-hooks, to make up the idle wheel assembly. The visual representation on Fig. 4 should clarify all elements that were difficult to explain concisely.

Figure 4. A complete continuous track.

The excavator arm and bucket were constructed entirely out of PLA filament, with aluminium rod bearings, servos, and a stepper motor all enabling movement. The individual arm pieces were modelled closely from construction excavators, with modifications made to ensure it could reach the required length. Each piece of the arm had a 20mm wide core, this was to allow the servos to be mounted without any hardware (i.e. friction fit). The servos were designed to push and pull on a separate set of "hydraulic" arms to move the entire assembly and mimic the actions of hydraulic cylinders (as seen prior on Fig. 3). Once all pieces were printed, shortened pieces of aluminium rod were inserted in all holes to hold the assembly together and allow rotation around them, which enabled unpowered movement. Servos were then disassembled and reassembled inside the cutouts in each arm and the base. The "hydraulic" arms were then mounted to the servos and attached to their corresponding rod bearing, enabling powered movement.

For the base of the excavator arm, two pieces were made, one to mount to the upper level of the chassis, and one to mount to the excavator assembly. These would be held together by their design, with a smaller cylinder on the chassis-side sitting inside a larger, hollow cylinder on the arm-side, with the inside diameter of the large, hollow cylinder being within 0.5mm of the outside diameter of the smaller cylinder. Inset on the chassis-side was a small stepper motor, whose drive shaft would extend into the arm-side of the base to allow rotation. Sketches of all components, including a full assembly rendering can be found in Appendix A.

Due to many last-minute changes, a full electrical diagram could not be provided for the device, however a brief description that should give enough of an understanding is as follows:

2S Battery→fuse→SPST switch→splitter.

Splitter A→buck converter (set to 5V)→5V rail of breadboard. This would power all UNO R3s, one L298N motor driver for the stepper motor, and all three servos on the excavator arm.

Splitter B→buck converter (set to 12V)→splitter→2x L298N motor drivers, these would drive the four motors of the continuous tracks.

These would all be grounded to the negative terminal of our battery pack.

I did not write the code, and so I cannot comment on the design process. For completeness I have, however, included all code for the device (see Appendix B).

Unfortunately, due to a necessary size change of the arm, the rock depositing mechanism no longer fitted within the size constraints of the machine and thus, had to be omitted from the final device. While the work done does not go unnoticed by the group, I have decided to omit a description of the manufacturing of the mechanism for the sake of brevity.

The complete first prototype device (pictured in Fig. 5) is a culmination of all the separate segments detailed in this section.

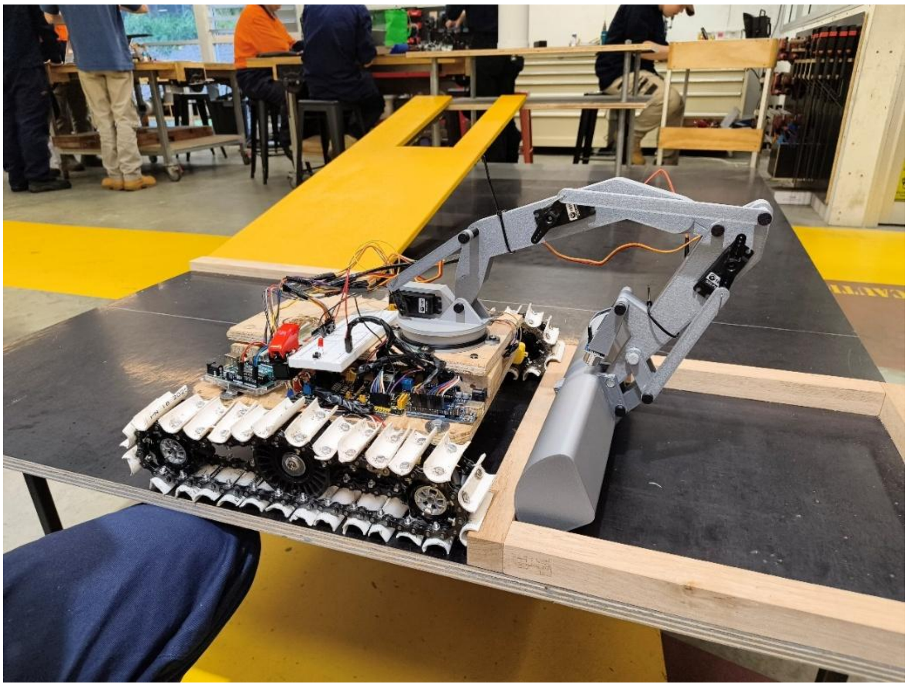


Figure 5. Completed first prototype Warman Device.

Final Solution and Evaluation

Final Prototype

For our final device, much of what was conceptualised from the start remained the same, the major difference being the omission of the rock deposit mechanism as mentioned above. Unfortunately, due in part to our inexperience (and no shortage of bad luck), many of our components failed, and thus, we were forced to make several modifications on the day of the competition. These modifications were as follows (and can be seen in Fig. 6):

- Switching the 12V power into the full 14.8V source power due to a fried buck converter. Buck converter failed due to unchecked wiring that ended up running power through the converter with no output from the converter.
- Rewiring the ground to use higher gauge wire as previous wire had melted.
- Modification of the excavator arm base to remove the stepper motor as the stepper motor could not rotate the arm assembly and attempts to remedy this further damaged the base attachment of the arm.
- Relocation of the excavator arm and removal of the upper level of the device to meet size constraints (also due to the failed stepper motor).
- Reversal of bucket direction (to allow new modifications to work).
- Printing of new "hydraulic" arms (to allow new bucket direction to work).
- Major code modifications to allow all prior changes to work.

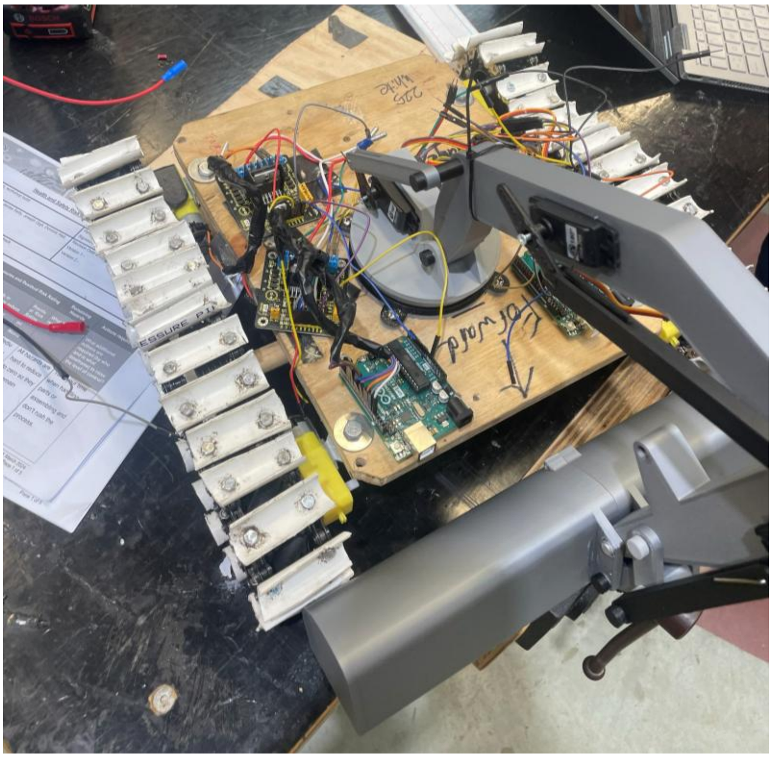


Figure 6. Final Prototype (after heavy modifications).

Pre-Competition Testing

Many tests were run on separate individual systems, the tracks consistently started drove straight in all tests and was able to successfully climb the test track ramp. While overvolting the motors did result in several burnt-out motors, we had many replacement motors on standby to swap in, so this was not an issue. Throughout the design and refinement of the excavator arm, many unpowered tests were run to test the capabilities of the current design, with the first fully powered test happening the night before the competition and yielding perfect results and performance. Video evidence of all systems working over the last few days were presented to the evaluator as evidence of the device being functional. (Shown here: https://youtu.be/6J6rn9JX0r4)

Day of Competition Testing and Evaluation

Unfortunately, due to the modifications made, we were unable to present a fully functioning prototype. No rocks were collected and the tracks failed to drive on the day, despite working well in the lead up to testing. While this did result in a technical failure, our pre-competition testing gave us confidence that, had nothing major malfunctioned in the days leading up to the test, we would have deposited all rocks on the upper level of the track. So, despite a competition day failure, our device overall presented a successful and novel design based on sound engineering principals that performed well in pre-competition testing. Nevertheless, much was learned in the design and testing of this device that will serve us well in future competitions.

Self-Reflection

Many of the problems with our device can be directly traced back to poor project management. When our group first met up, we made substantial progress in producing preliminary designs, but only two members in the group continued making progress throughout the following weeks. These two group members also failed to delegate tasks well to the other group members which left the other group members feeling aimless, leading to a lack of progress. The result of this was the 3 weeks leading up to competition, most of our group members worked 8 or more hours per day on the device, and while significant progress was made in this time, it left us with almost no time to finalise code or do a single mock run of the course. We also no longer had the time to order cheaper electrical components from overseas, which – along with the many failed costly components – greatly inflated our total cost. This culminated in the technical failure on the day of evaluation.

A more thorough reading of the rules by all team members would have greatly improved the final product also. Many of us had only skimmed the rules and did not have a deep enough knowledge to realise the inefficiencies that would be caused by making everything by hand. Particularly, the tracks took much time and money to make, and while testing had shown promise, I had doubts about the consistency of friction driven tracks. Pre-made continuous track systems would have been far faster and cheaper to purchase, and being gear driven, would be far less susceptible to slippage.

Component choice could have also been significantly improved upon. Several components were under-spec'd, which caused some of the competition day failures. We should have checked calculations on stepper motor torque instead of blindly trusting that our mental math was current. Similarly, while DC motors appeared to work well for the tracks, stepper motors may have been a better choice and would certainly be something I would change if we were tasked with this challenge again.

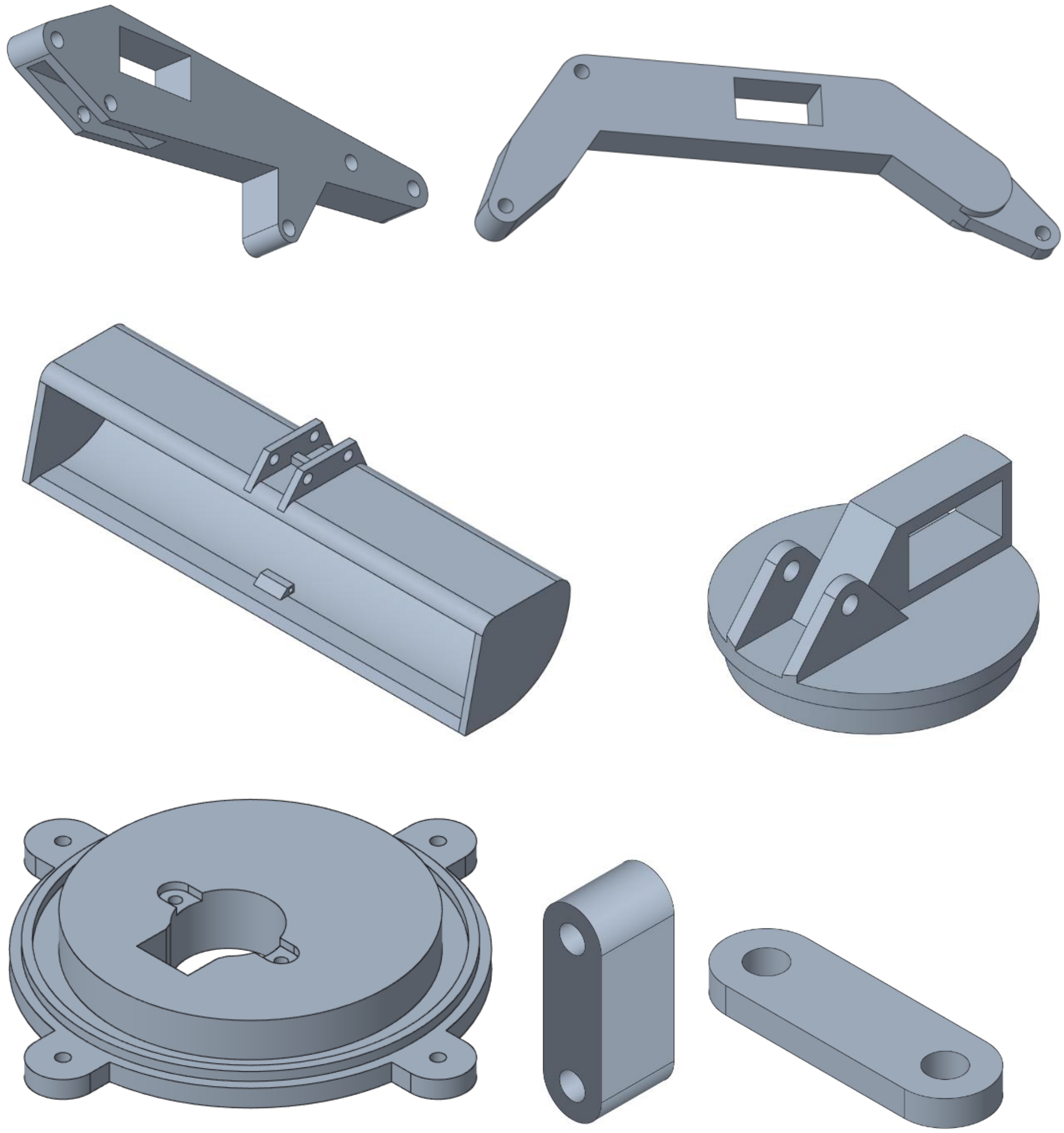
Another, more minor failure point for the team was the late addition of a team-member. Due to the expectation that we would only be working with four group members, task delegations were already made, which gave our new team member little to do. This failure on the team's part led to that group member putting in considerably less of work. Pre-existing commitments of some of the team members also slowed progress in those crucial 3 weeks leading up to the test day. This issue was completely avoidable had we put more time and effort into project management early.

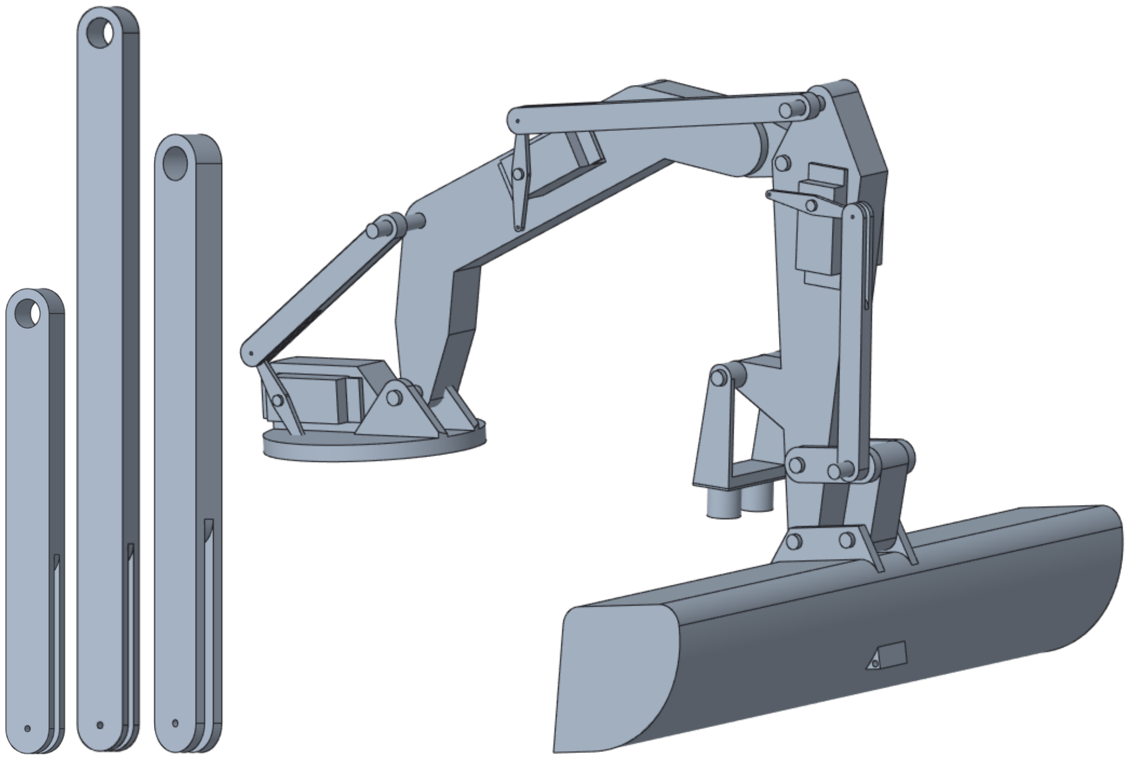
Conclusion

Overall, this competition was a brilliant lesson in project management and engineering practices, while maintaining the fun of a practical engineering project. Our team set out to create a device capable of collecting rocks, transporting them, and depositing them on the upper platform. While our device demonstrated solid engineering principals – with both the continuous tracks and the excavator arm operating successfully, albeit independently – component failures and last-minute modifications prevented a successful test day run. They did, however, teach the valuable lesson of time management, deadlines, and early testing.

While our design was far from perfect, it appeared sound enough to be able to complete the assigned task, our issues were rooted primarily in project management shortcomings. Uneven workload distribution, delayed progress, and insufficient research on both components and the rules compressed our timeline to the point where early testing became impossible. Future iterations would benefit from clear task delegation, earlier and stricter deadlines, and a milestone-based approach to the project. Stricter component selection with better justification and calculations would have greatly increased the likelihood of first try success. With these improvements, the overall concept has a strong foundation and could realistically be extended to re-include the deposit mechanism to pursue a higher-scoring test day. Ultimately, the final demonstration did not meet the competition, nor our own, objectives. However, it did ensure that future projects of mine will include structed project management and tighter deadlines, as I have now learnt these skills as a direct result of this course.

Appendix A





Appendix B

Arduino 1:

```
#include <Servo.h>
#include <Wire.h>

#define SLAVE_ADDR 9
bool slaveDone = false;
int stages = 0;

// RHS

int IN1R = 2;
int IN2R = 3;
int IN3R = 4;
int IN4R = 5;
int ENAR = 6;
int ENBR = 7;

// LHS

int IN1L = 8;
int IN2L = 9;
int IN3L = 10;
int IN4L = 11;
int ENAL = 12;
int ENBL = 13;


void sendCommand(const char* cmd) {
  Wire.beginTransmission(SLAVE_ADDR);
  Wire.write(cmd);
  Wire.endTransmission();
}

String requestResponse() {
  String response = "";

  Wire.requestFrom(SLAVE_ADDR, 10);  //requesting up to 10 bytes, 10
characters??

  while (Wire.available()) {
    char c = Wire.read();
    response += c;
```

```
  }
  return response;
  Serial.println("Got response");
}

void setup() {
  Wire.begin();
  Serial.begin(9600);
  pinMode(IN1R, OUTPUT);
  pinMode(IN2R, OUTPUT);
  pinMode(IN3R, OUTPUT);
  pinMode(IN4R, OUTPUT);
  pinMode(ENAR, OUTPUT);
  pinMode(ENBR, OUTPUT);

  pinMode(IN1L, OUTPUT);
  pinMode(IN2L, OUTPUT);
  pinMode(IN3L, OUTPUT);
  pinMode(IN4L, OUTPUT);
  pinMode(ENAL, OUTPUT);
  pinMode(ENBL, OUTPUT);
}



void loop() {
  switch (stages) {
    case (0):;
      {
        sendCommand("BUTTON");
        Serial.println("Awaiting Start");
        slaveDone = false;

        while (!slaveDone) {
          String response = requestResponse();

          if (response == "DONE")
            ;
          slaveDone = true;
          Serial.println("Ready Position Complete");
          delay(100);
          stages++;
          stages++;
        }
      }
    case (1):;
```

```
    {  // send message to servos for collection
      sendCommand("START");
      Serial.println("Ready position");
      slaveDone = false;

      while (!slaveDone) {
        String response = requestResponse();

        if (response == "DONE")
          ;
        slaveDone = true;
        Serial.println("Ready Position Complete");
        delay(100);
        stages++;
      }
    }
  case (2):;
    {  // arm extension in dump zone
      sendCommand("Extend");
      Serial.println("Extending");
      slaveDone = false;
      while (!slaveDone) {
        String response = requestResponse();

        if (response == "DONE")
          ;
        slaveDone = true;
        Serial.println("Extension Complete");
        delay(100);
        stages++;
      }
    }

  case (3):;
    {  // bucket tilt up
      sendCommand("BUCKETUP");
      Serial.println("BUCKET UP");
      slaveDone = false;
      while (!slaveDone) {
        String response = requestResponse();

        if (response == "DONE")
          ;
        slaveDone = true;
        Serial.println("Rocks are high");
        delay(100);
```

```
        stages++;
      }
    }
  case (4)::
    {  //backwards movement
      Serial.println("Moving away from dump zone");
      backwards();
      analogWrite(ENAR, 150);
      analogWrite(ENBR, 150);
      analogWrite(ENAL, 150);
      analogWrite(ENBL, 150);
      delay(500); ////////////////////////////////
      stop();
      delay(100);
      stages++;
    }

  case (5)::
    {  //turning
      Serial.println("Turning");
      lturn();
      analogWrite(ENAR, 150);
      analogWrite(ENBR, 150);
      analogWrite(ENAL, 150);
      analogWrite(ENBL, 150);
      delay(1100); //////////////////////////////
      stop();
      delay(100);
      stages++;
    }
  case (6)::
    {  // going up ramp
      forwards();
      analogWrite(ENAR, 150);
      analogWrite(ENBR, 150);
      analogWrite(ENAL, 150);
      analogWrite(ENBL, 150);
      delay(3000); ////////////////////////////
      stop();
      delay(100);
      stages++;
    }
  case (7)::
    {  // bucket dump
      sendCommand("DUMP");
      Serial.println("DUMPING");
```

```
        slaveDone = false;
        while (!slaveDone) {
          String response = requestResponse();

          if (response == "DONE")
            ;
          slaveDone = true;
          Serial.println("Rocks are down, I repeat, rocks are down");
          delay(100);
          stages++;
        }
      }
    case (8):;
      {  // going up ramp
        forwards();
        analogWrite(ENAR, 150);
        analogWrite(ENBR, 150);
        analogWrite(ENAL, 150);
        analogWrite(ENBL, 150);
        delay(2000);  ////////////////////////////
        stop();
        delay(100);
        stages++;
      }
    case (9):;
      {  // doing down the ramp
        backwards();
        analogWrite(ENAR, 100);
        analogWrite(ENBR, 100);
        analogWrite(ENAL, 100);
        analogWrite(ENBL, 100);
        delay(2000);
        stop();
        delay(100);
        stages++;
      }
    case (10):;
      {
        // led
      }
  }
}


void forwards() {
  digitalWrite(IN1R, HIGH);
```

```
  digitalWrite(IN2R, LOW);

  digitalWrite(IN3R, HIGH);
  digitalWrite(IN4R, LOW);

  digitalWrite(IN1L, LOW);
  digitalWrite(IN2L, HIGH);

  digitalWrite(IN3L, LOW);
  digitalWrite(IN4L, HIGH);
}

void backwards() {
  digitalWrite(IN1R, LOW);
  digitalWrite(IN2R, HIGH);

  digitalWrite(IN3R, LOW);
  digitalWrite(IN4R, HIGH);

  digitalWrite(IN1L, HIGH);
  digitalWrite(IN2L, LOW);

  digitalWrite(IN3L, HIGH);
  digitalWrite(IN4L, LOW);
}
void stop() {
  digitalWrite(IN1R, LOW);
  digitalWrite(IN2R, LOW);

  digitalWrite(IN3R, LOW);
  digitalWrite(IN4L, LOW);

  digitalWrite(IN1R, LOW);
  digitalWrite(IN2L, LOW);

  digitalWrite(IN3R, LOW);
  digitalWrite(IN4L, LOW);
  analogWrite(ENAR, 0);
  analogWrite(ENBR, 0);
  analogWrite(ENAL, 0);
  analogWrite(ENBL, 0);
}

void lturn() {
  digitalWrite(IN1R, LOW);
  digitalWrite(IN2R, HIGH);
```

```
  digitalWrite(IN3R, LOW);
  digitalWrite(IN4R, HIGH);

  digitalWrite(IN1L, LOW);
  digitalWrite(IN2L, HIGH);

  digitalWrite(IN3L, LOW);
  digitalWrite(IN4L, HIGH);
}
```

Arduino 2:

```
#include <Servo.h>
#include <Wire.h>

#define SERVOPIN 8  // servo pins for excavator arm
#define SERVOPIN 9
#define SERVOPIN 10

int IN1 = 7;
int IN2 = 6;
int IN3 = 5;
int IN4 = 4;
int ENA = 3;
int ENB = 2;

Servo bucket;   //pos1
Servo middle;   //pos2
Servo boom;     //pos3

int trigPin = 11;  // ultrasonic sensor pins for bucket
int echoPin = 12;
unsigned long previousMillis = 0;
const long interval = 60; // delay between readings

float distanceMM;                  // variable for ultrasonic calculation
long duration;                     // variable for ultrasonic calculation
int currentAngle = middle.read();  // for the middle servo to be able to
adjust a variable

int stages = 0;      // for switches to go from rock collection, to ramp,
to finish, etc
int collection = 0;  // for rock collection
```

```
int speed = 30;      // speed value for our middle servo



void excavator_extension() {
  // void defines the function, to call we need the name of the functipn
"excavator_extension"
  // followed by ()
  bucket.write(180);  // bucket 180 so its flat against surface  // need
to define an extension to run this so it can just be called on. Also
  middle.write(90);   //
  boom.write(90);
  delay(1000);
}

void excavator_extension2() {  // full distance
  bucket.write(180);           // could put delays in middle if needed
  middle.write(180);
  boom.write(180);
  delay(1000);
}

void boom_drawback() {
  for (int i = 180; i >= 0; i--) {  //this will bring the middle
connection back at a set rate by moving through each step/degree
                                    //could also use the VarSpeedServo
library, myServo.slowmove(180,30) / moves 180 degrees at a rate of 30
    boom.write(i);
    delay(50);  //increasing this delay will make it slower
    // i can just add i values into the running code as the untrasonic
sensor picks up the ground
    // if US distance is >10 i--, might need to use Var Speed so i can
slow the motor
  }
}

void ultrasonic_sensor() {
  // send trig to HIGH to send out an ultrasonic burst
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);  //10 is lowest
  digitalWrite(trigPin, LOW);

  //read the echo pin, pulseIn() returns the travel time in microseconds
  duration = pulseIn(echoPin, HIGH);

  //calculation distance
```

```
  // Speed of sound - 0.034 cm/uS or 0.343 mm/uS
  distanceMM = duration * 0.343 / 2;  // the /2 is for the distance out
and back

  Serial.print("Distance: ");
  Serial.print(distanceMM);
  Serial.println(" mm");
}

void middle_drawback() {
  float dist = distanceMM;
  if (dist > 200) {
    currentAngle += 1;
  } else if (dist < 100) {
    currentAngle -= 1;
  }
}

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  bucket.attach(8);  //bucket pin number
  middle.attach(9);
  boom.attach(10);

  bucket.write(0);  // set servos to there related 0 value, values will
need to be adjusted accordingly
  middle.write(0);
  boom.write(0);
  delay(100);

  pinMode(trigPin, OUTPUT);  // sets trig as output and echo as input.
trig sends out sound, echo recieves it
  pinMode(echoPin, INPUT);
  digitalWrite(trigPin, LOW);  //clear the trigger pin by setting it to
low for a clean pulse
  delayMicroseconds(2);
}

void loop() {
  switch (stages) {
    case (0):;  /// getting to rock collection
      {
        stages++;
      }
      break;
```

```
    case (1):;  // rock collection
      {
        switch (collection) {
          case (0):;  //extension
            {
              Serial.println("Extension 1");
              excavator_extension();
              delay(100);

              collection++;
            }
            break;

          case (1):;  //pulling back
            {
              Serial.println("Drawback 1");
              ultrasonic_sensor();
              boom_drawback();
              middle_drawback();           //adjusts current angle
              middle.write(currentAngle);  // moves servo
              delay(100);

              collection++;
            }
            break;

          case (2):;
            {  // lifting up, could use either mapping or just sent servo
angles with for to make it smooth
              collection++;
            }
            break;

          case (3):;
            {  // rotating around + dump + rotate back
              collection++;
            }
            break;

          case (4):;
            {  //second extension
              excavator_extension2();
              delay(100);

              collection++;
            }
```

```
          break;

        case (5):;
        { // second drawback
          ultrasonic_sensor();
          boom_drawback();
          middle_drawback();            //adjusts current angle
          middle.write(currentAngle);   // moves servo
          delay(100);

          collection++;
        }
        break;

        case(6):;
        { // second pick up
          //
        }
        break;

        case(7):;
        { // second rotate + drop + finish position
          //
        }
        break;
      }
    }
  }
}
```

Bibliography

[1] Engineers Australia, “39th Warman Design and Build Competition Rules,” Engineers Australia, 2026. https://warmandesignandbuild.org.au/

[2] M. Fiedeń and J. Bałchanowski, “A Mobile Robot with Omnidirectional Tracks—Design and Experimental Research,” Applied Sciences, vol. 11, no. 24, p. 11778, Dec. 2021, doi: https://doi.org/10.3390/app112411778.

[3] HR Parts, “How Does an Excavator Arm Work | Basic Details, Images, and Animated Diagram,” www.hrparts.com, Oct. 26, 2022. https://www.hrparts.com/blog/post/how-does-excavator-arm-work-diagram